

```
!pip install -q xgboost scikit-learn pandas numpy matplotlib seaborn
```

```
from google.colab import files
uploaded = files.upload() # upload: rainfall.csv, temp.csv, pesticides.csv, yield.csv
```

Choose Files 5 files

```
pesticides.csv(text/csv) - 447677 bytes, last modified: 7/6/2026 - 100% done
rainfall.csv(text/csv) - 136712 bytes, last modified: 7/6/2026 - 100% done
temp.csv(text/csv) - 1421414 bytes, last modified: 7/6/2026 - 100% done
yield.csv(text/csv) - 4042934 bytes, last modified: 7/6/2026 - 100% done
yield_df.csv(text/csv) - 1565376 bytes, last modified: 7/6/2026 - 100% done
Saving pesticides.csv to pesticides.csv
Saving rainfall.csv to rainfall.csv
Saving temp.csv to temp.csv
Saving yield.csv to yield.csv
Saving yield_df.csv to yield_df.csv
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from xgboost import XGBRegressor
```

```
sns.set_style("whitegrid")
np.random.seed(42)
```

```
rain = pd.read_csv("rainfall.csv")
rain.columns = [c.strip() for c in rain.columns]
rain["Area"] = rain["Area"].str.strip()
# "." placeholders hide as text -- force numeric conversion
rain["average_rain_fall_mm_per_year"] = pd.to_numeric(
    rain["average_rain_fall_mm_per_year"], errors="coerce"
)

temp = pd.read_csv("temp.csv").rename(columns={"year": "Year", "country": "Area"})

pest = pd.read_csv("pesticides.csv")[["Area", "Year", "Value"]]\
    .rename(columns={"Value": "pesticides_tonnes"})

yld = pd.read_csv("yield.csv")
yld = yld[yld["Element"] == "Yield"][["Area", "Item", "Year", "Value"]]\
    .rename(columns={"Value": "hg/ha_yield"})

crops_10 = ["Maize", "Potatoes", "Rice, paddy", "Sorghum", "Soybeans", "Wheat",
            "Cassava", "Sweet potatoes", "Plantains and others", "Yams"]
yld = yld[yld["Item"].isin(crops_10) & yld["Year"].between(1990, 2013)]

df = yld.merge(rain, on=["Area", "Year"], how="left")
df = df.merge(temp, on=["Area", "Year"], how="left")
df = df.merge(pest, on=["Area", "Year"], how="left")
df["yield_tonnes_per_ha"] = df["hg/ha_yield"] / 10000.0
target = "yield_tonnes_per_ha"

numeric_gap_cols = ["average_rain_fall_mm_per_year", "avg_temp", "pesticides_tonnes"]
```

```
for c in numeric_gap_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")

print("Merged shape (before cleaning):", df.shape)
df.head()
```

Merged shape (before cleaning): (50051, 8)

	Area	Item	Year	hg/ha_yield	average_rain_fall_mm_per_year	avg_temp	pesticides_tonne
0	Afghanistan	Maize	1990	17582	327.0	15.45	Na
1	Afghanistan	Maize	1991	16800	327.0	14.57	Na
2	Afghanistan	Maize	1992	15000	327.0	14.35	Na
3	Afghanistan	Maize	1993	16786	327.0	14.96	Na
4	Afghanistan	Maize	1994	16667	327.0	14.94	Na

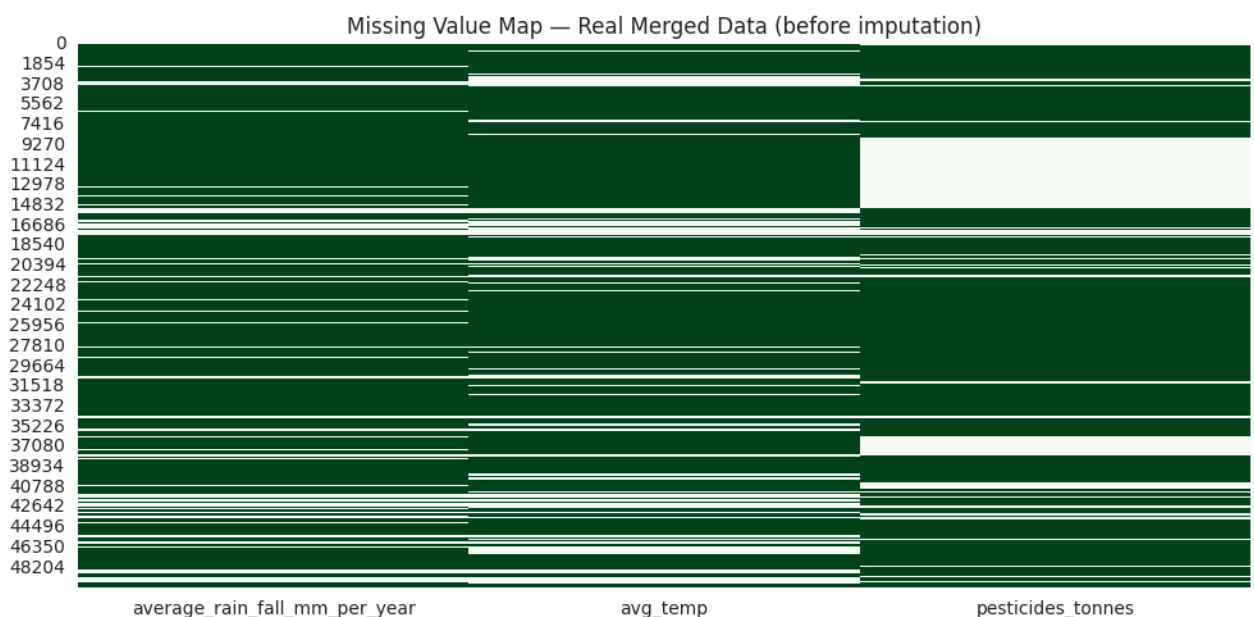
Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
missing_before = df[numeric_gap_cols].isnull().sum()
missing_pct_before = (missing_before / len(df) * 100).round(1)
print(pd.DataFrame({"Missing_Count": missing_before, "Missing_%": missing_pct_before}))
```

	Missing_Count	Missing_%
average_rain_fall_mm_per_year	7332	14.6
avg_temp	10243	20.5
pesticides_tonnes	12860	25.7

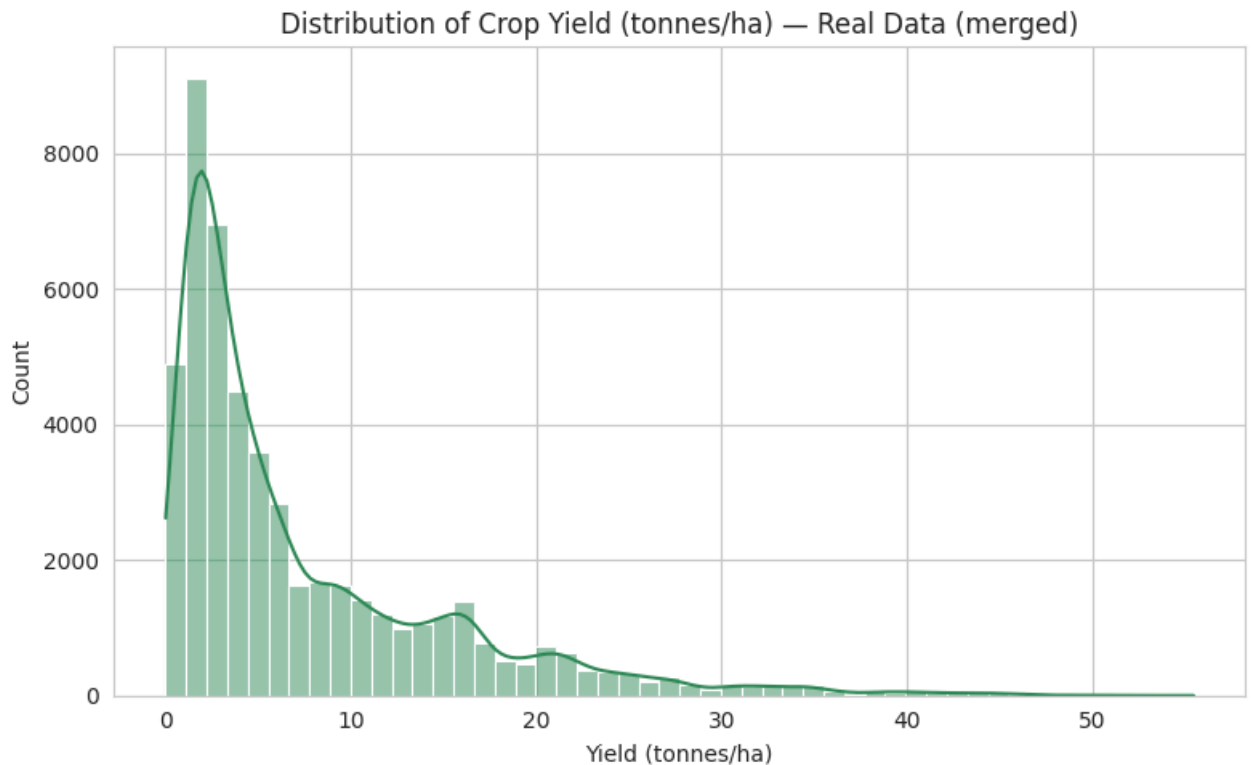
```
plt.figure(figsize=(10, 5))
sns.heatmap(df[numeric_gap_cols].isnull(), cbar=False, cmap="Greens_r")
plt.title("Missing Value Map – Real Merged Data (before imputation)")
plt.tight_layout()
plt.savefig("fig_missing_map.png", dpi=150)
plt.show()
```



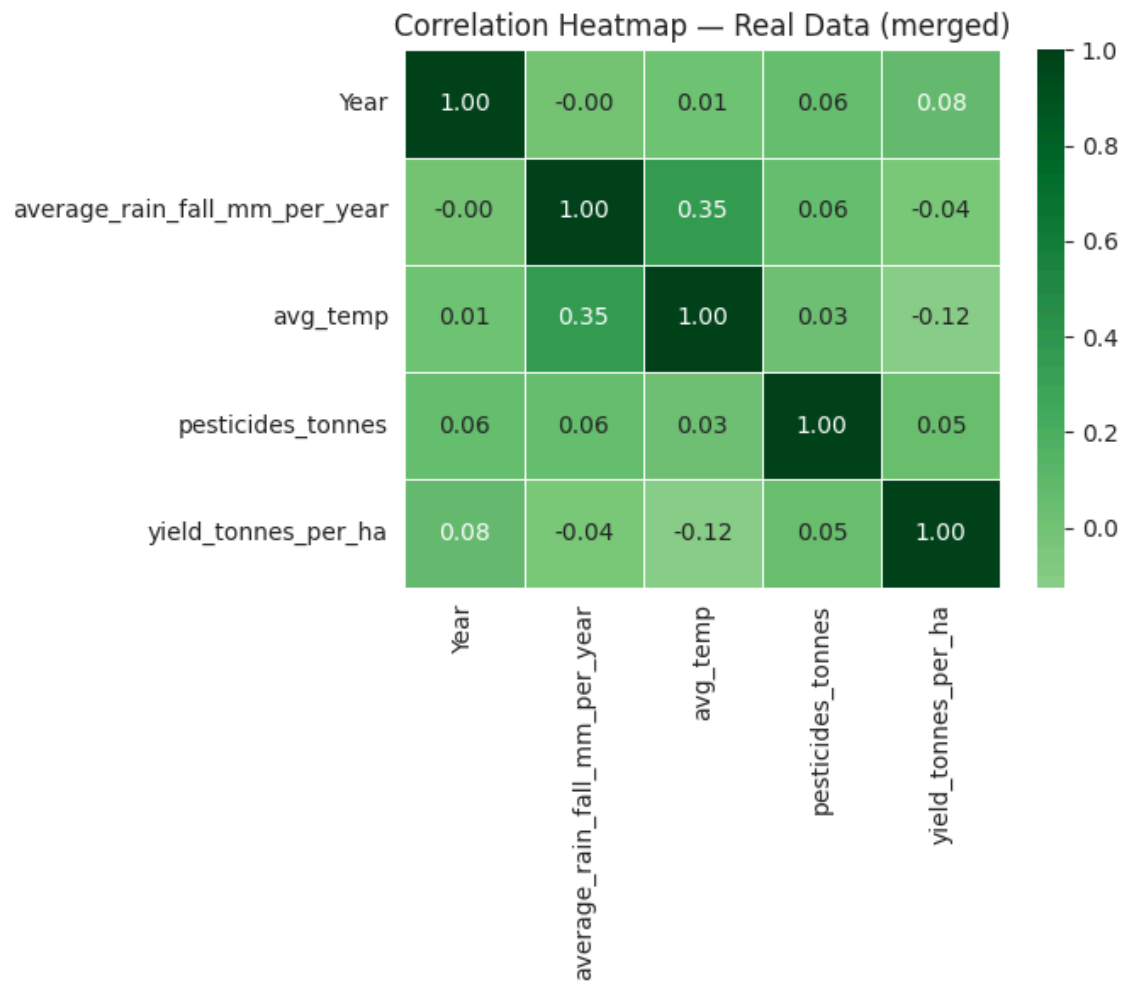
```
imputer = SimpleImputer(strategy="median")
df[numeric_gap_cols] = imputer.fit_transform(df[numeric_gap_cols])
print("Missing values after imputation:", df.isnull().sum().sum())
```

Missing values after imputation: 0

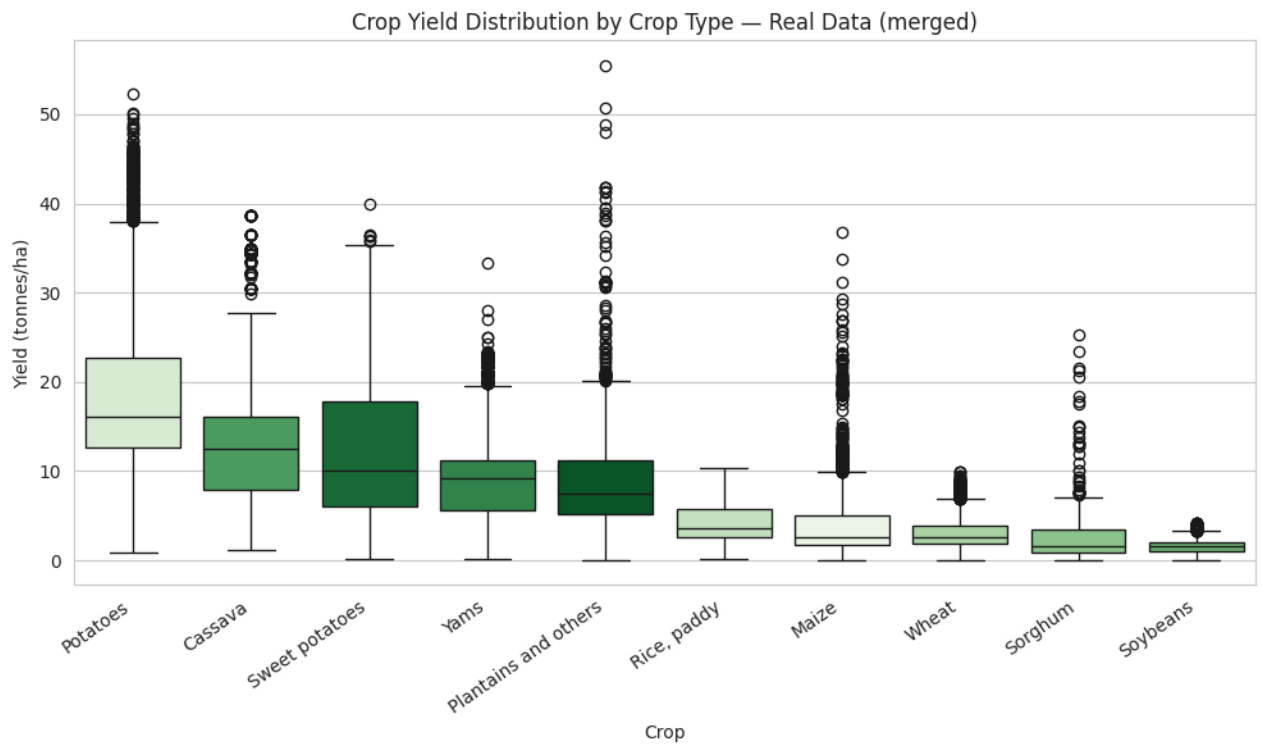
```
plt.figure(figsize=(8, 5))
sns.histplot(df[target], kde=True, bins=50, color="seagreen")
plt.title("Distribution of Crop Yield (tonnes/ha) – Real Data (merged)")
plt.xlabel("Yield (tonnes/ha)")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("fig1_distribution.png", dpi=150)
plt.show()
```



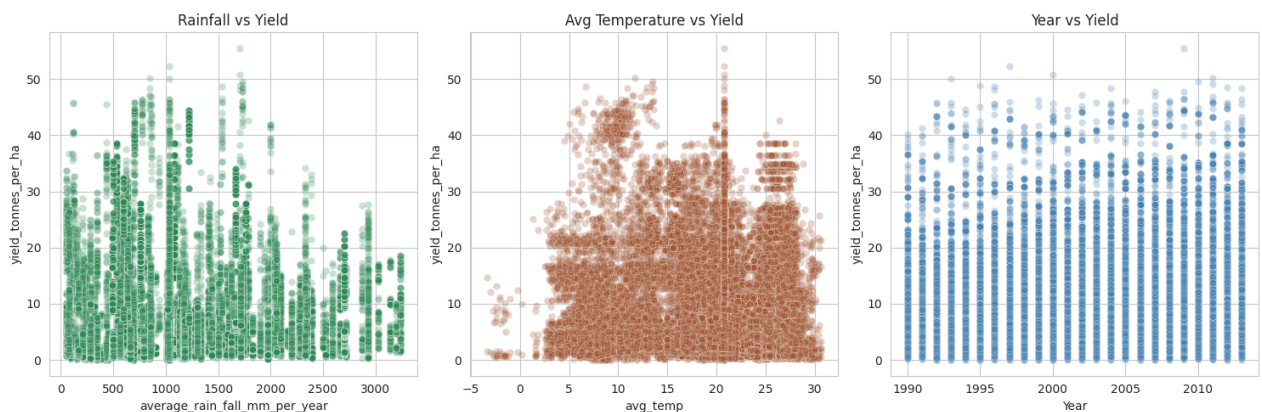
```
plt.figure(figsize=(7, 6))
num_cols = ["Year"] + numeric_gap_cols + [target]
corr = df[num_cols].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="Greens", center=0, linewidths=0.5)
plt.title("Correlation Heatmap – Real Data (merged)")
plt.tight_layout()
plt.savefig("fig2_correlation.png", dpi=150)
plt.show()
```



```
plt.figure(figsize=(10, 6))
order = df.groupby("Item")[target].median().sort_values(ascending=False).index
sns.boxplot(data=df, x="Item", y=target, order=order, hue="Item",
            palette="Greens", legend=False)
plt.title("Crop Yield Distribution by Crop Type – Real Data (merged)")
plt.xlabel("Crop")
plt.ylabel("Yield (tonnes/ha)")
plt.xticks(rotation=35, ha="right")
plt.tight_layout()
plt.savefig("fig3_yield_by_crop.png", dpi=150)
plt.show()
```



```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
sns.scatterplot(data=df, x="average_rain_fall_mm_per_year", y=target,
                alpha=0.25, ax=axes[0], color="seagreen")
axes[0].set_title("Rainfall vs Yield")
sns.scatterplot(data=df, x="avg_temp", y=target,
                alpha=0.25, ax=axes[1], color="sienna")
axes[1].set_title("Avg Temperature vs Yield")
sns.scatterplot(data=df, x="Year", y=target,
                alpha=0.25, ax=axes[2], color="steelblue")
axes[2].set_title("Year vs Yield")
plt.tight_layout()
plt.savefig("fig4_climate_drivers.png", dpi=150)
plt.show()
```



```
features_num = ["Year"] + numeric_gap_cols
features_cat = ["Area", "Item"]

train_df = df[df["Year"] <= 2008]
```

```

val_df = df[(df["Year"] > 2008) & (df["Year"] <= 2011)]
test_df = df[df["Year"] > 2011]

def encode(part, ref_columns=None):
    dummies = pd.get_dummies(part[features_cat], columns=features_cat)
    out = pd.concat([part[features_num].reset_index(drop=True),
                     dummies.reset_index(drop=True)], axis=1)
    if ref_columns is not None:
        out = out.reindex(columns=ref_columns, fill_value=0)
    return out

X_train = encode(train_df)
ref_cols = X_train.columns
X_val = encode(val_df, ref_cols)
X_test = encode(test_df, ref_cols)

y_train = train_df[target].values
y_val = val_df[target].values
y_test = test_df[target].values

print("Train:", X_train.shape, " Val:", X_val.shape, " Test:", X_test.shape)

```

Train: (39463, 224) Val: (6337, 224) Test: (4251, 224)

```

rf_model = RandomForestRegressor(
    n_estimators=300, max_depth=16, min_samples_split=4,
    min_samples_leaf=2, random_state=42, n_jobs=-1
)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))
rf_mae = mean_absolute_error(y_test, rf_pred)
rf_r2 = r2_score(y_test, rf_pred)
print(f"Random Forest -> RMSE: {rf_rmse:.3f} MAE: {rf_mae:.3f} R2: {rf_r2:.4f}")

```

Random Forest -> RMSE: 3.182 MAE: 1.754 R2: 0.8649

```

xgb_model = XGBRegressor(
    n_estimators=1000, max_depth=6, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0,
    random_state=42, n_jobs=-1, objective="reg:squarederror",
    early_stopping_rounds=30, eval_metric="rmse"
)
xgb_model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=False)
xgb_pred = xgb_model.predict(X_test)

xgb_rmse = np.sqrt(mean_squared_error(y_test, xgb_pred))
xgb_mae = mean_absolute_error(y_test, xgb_pred)
xgb_r2 = r2_score(y_test, xgb_pred)
print(f"XGBoost -> RMSE: {xgb_rmse:.3f} MAE: {xgb_mae:.3f} R2: {xgb_r2:.4f}")
print("Best iteration:", xgb_model.best_iteration)

```

XGBoost -> RMSE: 2.633 MAE: 1.517 R2: 0.9075
Best iteration: 999

```

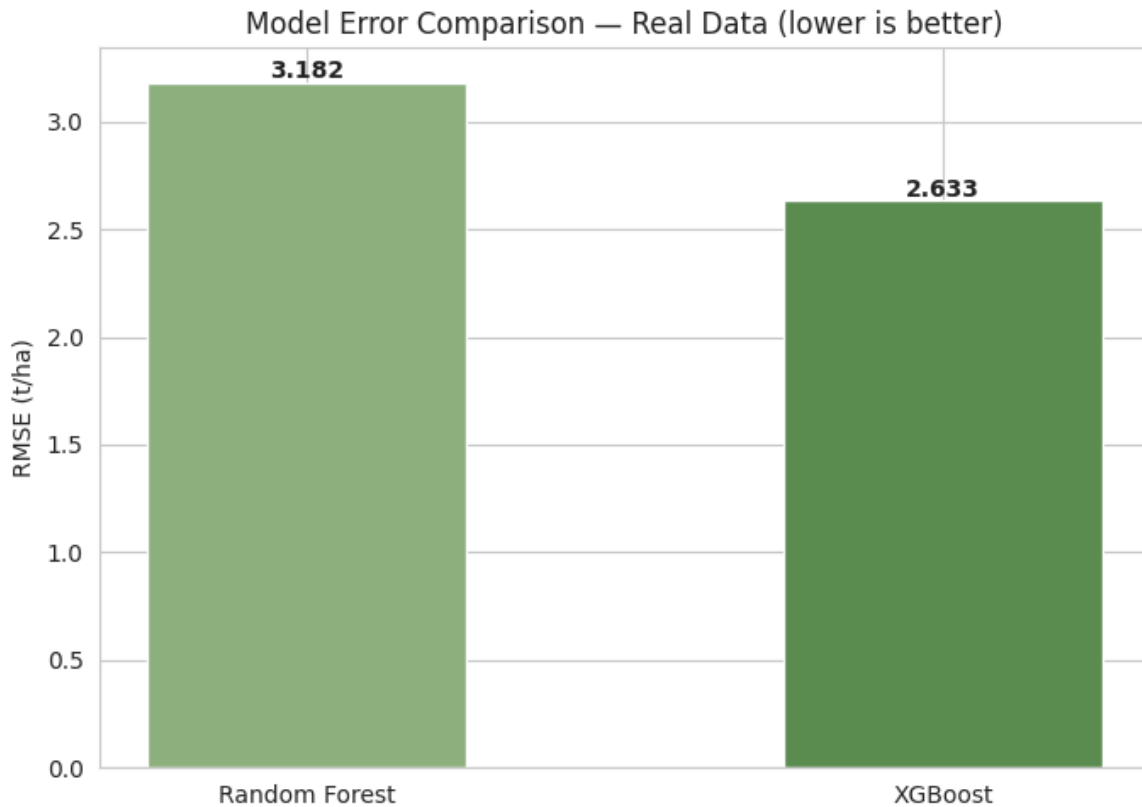
results = pd.DataFrame({
    "Model": ["Random Forest", "XGBoost"],
    "RMSE": [rf_rmse, xgb_rmse]
})

```

```

plt.figure(figsize=(7, 5))
bars = plt.bar(results["Model"], results["RMSE"], color=["#8FAF7E", "#5B8C51"], width=0.5)
plt.ylabel("RMSE (t/ha)")
plt.title("Model Error Comparison – Real Data (lower is better)")
for b, v in zip(bars, results["RMSE"]):
    plt.text(b.get_x()+b.get_width()/2, v+0.02, f"{v:.3f}", ha="center", fontweight="bold")
plt.tight_layout()
plt.savefig("fig5_rmse.png", dpi=150)
plt.show()

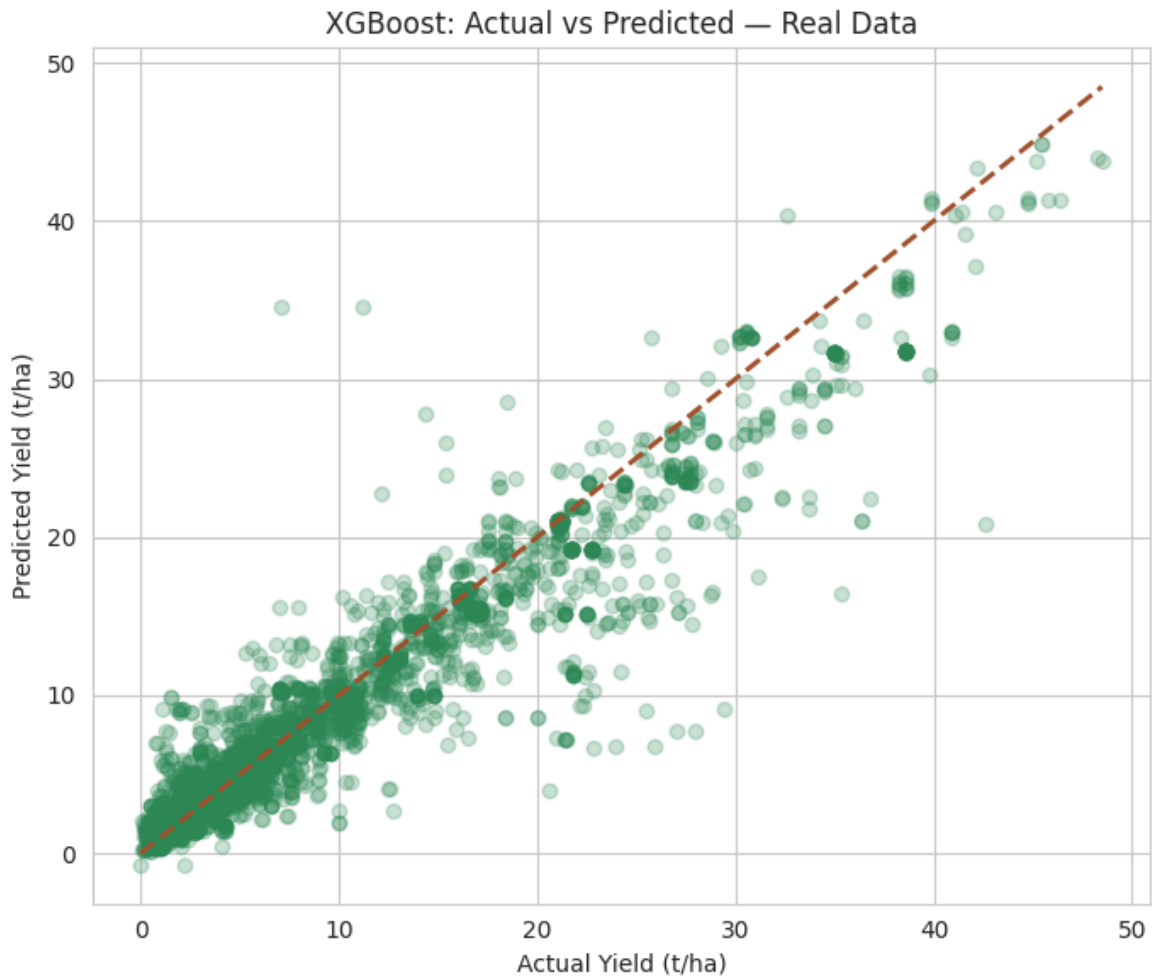
```



```

plt.figure(figsize=(7, 6))
plt.scatter(y_test, xgb_pred, alpha=0.25, color="seagreen")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "--", color="sienna", lw=2)
plt.xlabel("Actual Yield (t/ha)")
plt.ylabel("Predicted Yield (t/ha)")
plt.title("XGBoost: Actual vs Predicted – Real Data")
plt.tight_layout()
plt.savefig("fig6_actual_vs_predicted.png", dpi=150)
plt.show()

```



```
importance = pd.Series(xgb_model.feature_importances_, index=X_train.columns)\
                .sort_values(ascending=True).tail(8)
plt.figure(figsize=(7, 5))
plt.barh(importance.index, importance.values, color="seagreen")
plt.xlabel("Importance")
plt.title("Top Features Driving Yield – Real Data (XGBoost)")
plt.tight_layout()
plt.savefig("fig7_feature_importance.png", dpi=150)
plt.show()
```

